

Del Código a la Arquitectura

El Criterio del Desarrollador

Trabajo Final Integrador (TFI)

Materia: Desarrollo de Aplicaciones Web

Sistema de Gestión de Canchas y Matchmaking Deportivo

Resumen Ejecutivo

Este documento presenta el diseño arquitectónico completo de una plataforma de gestión de canchas deportivas y matchmaking, desarrollado como Trabajo Final Integrador de la materia Desarrollo de Aplicaciones Web. El proyecto recorre las cinco fases de evolución de un sistema real: desde un monolito MVC inicial hasta una arquitectura Event-Driven enriquecida con inteligencia artificial, cubriendo infraestructura cloud, DevOps y pipelines CI/CD.

FASE 1 Arquitectura Base — El MVP

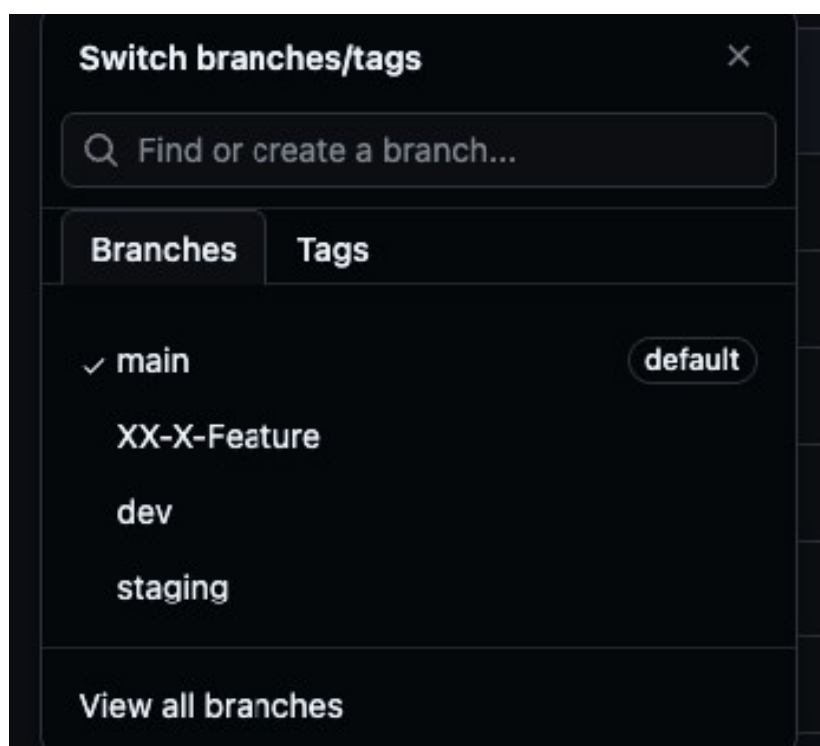
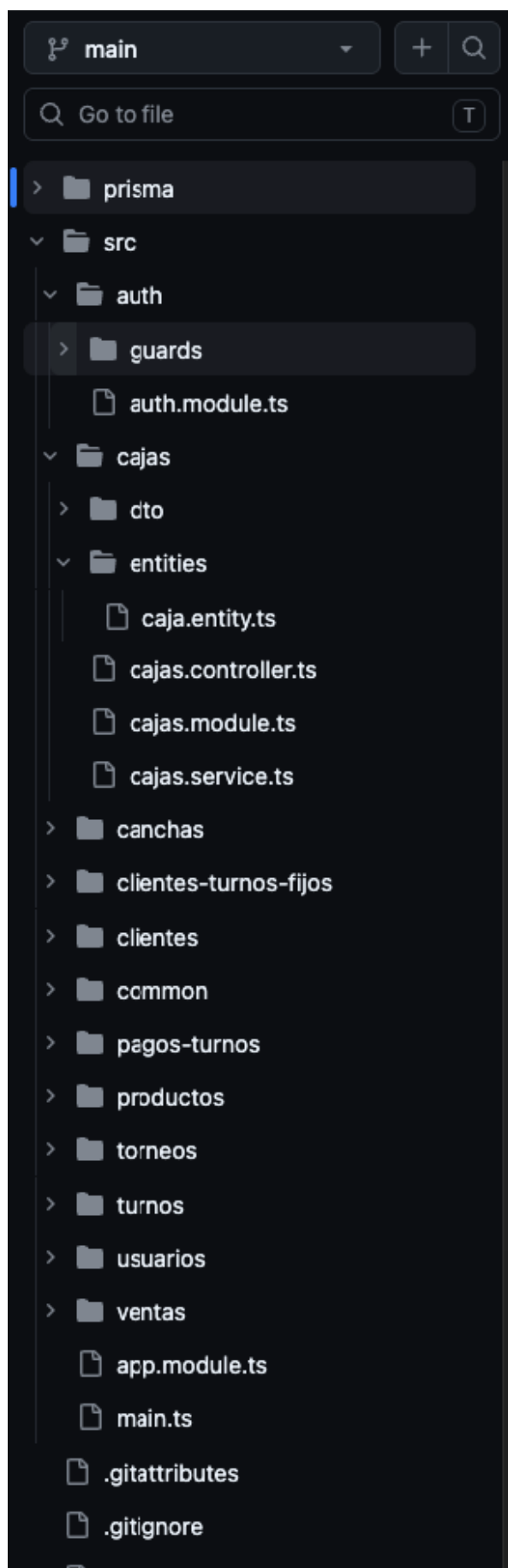
Descripción del sistema

El sistema de gestión de turnos y alquiler de canchas parte de una arquitectura monolítica modular en capas (MVC), apropiada para la etapa inicial del negocio. La lógica de negocio —disponibilidad de canchas, reservas y pagos— convive en una única unidad desplegable que accede a una base de datos relacional (PostgreSQL). El negocio presenta actualmente una sola cancha, con una sola base de datos de productos, reservas, clientes y ventas, se eligió esta arquitectura para reducir la complejidad operacional y poder entregar al cliente un sistema funcionando en optimas condiciones en base a su negocio actual, sin embargo, se optó por hacerlo modular para poder facilitar en un futuro la evolución a microservicios.

Características de la arquitectura inicial

- Stack: NestJS + PostgreSQL (Prisma ORM) + React en frontend
- Patrón MVC con separación de capas: Controladores, Servicios y Repositorios
- Autenticación con JWT; gestión de sesiones server-side

Diagrama de captura inicial y Branch strategy:



FASE 2 Evolución Forzada — Motor de Matchmaking

El escenario de quiebre

La evolución a microservicios se realiza debido a la necesidad técnica respecto al incremento de la cantidad de usuarios del sistema, la posibilidad de tener varios locales con varios productos

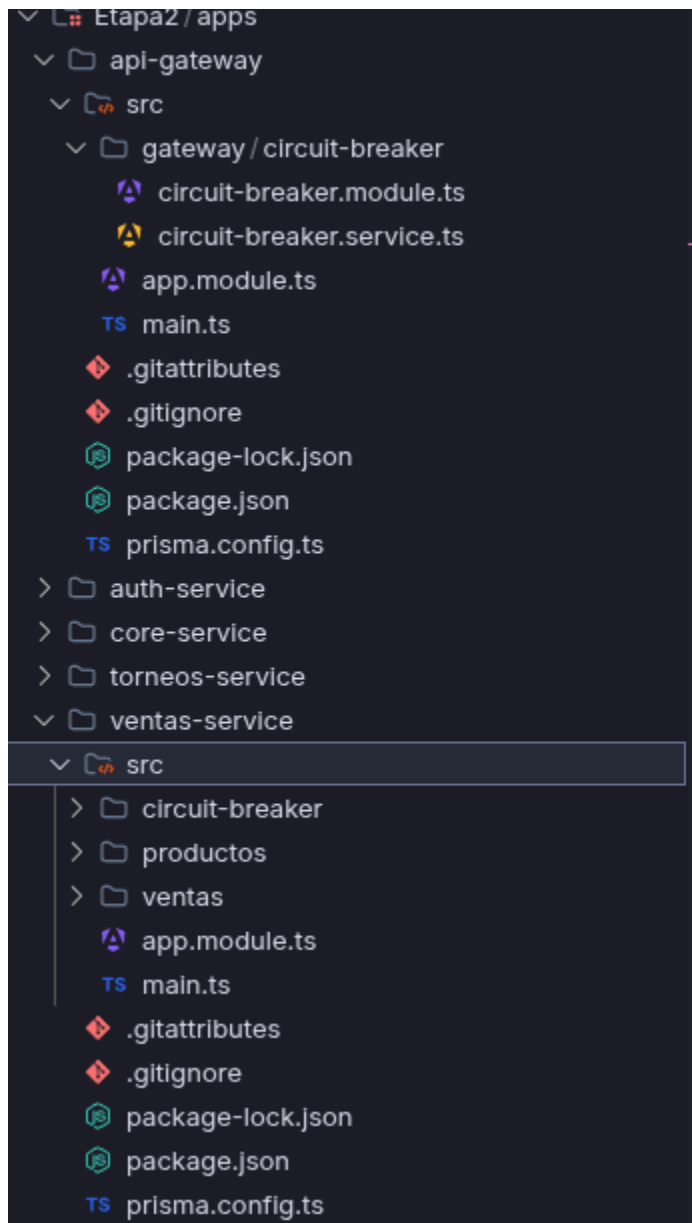
Al haber más cantidad de usuarios, más cantidad de gente y por ende mucho más tráfico, el monolito presenta una ralentización a la hora de realizar consultas a la base de datos y bloqueando la misma.

Este escenario se hace presente sobre todo en los horarios de mayor demanda de la aplicación.

Paso A — Microservicios con HTTP/REST sincrónico

Como primera iteración, se separan todos los modules y se los agrupa en los siguientes servicios. Auth-Service (incluyendo modulo Auth y Usuarios), core-service (incluyendo cajas, canchas, clientes, clientes-turnos-fijos, pagos-turnos, turnos), ventas-service (incluyendo circuit-breaker, productos, ventas), torneos-service (incluyendo modulo de torneos)

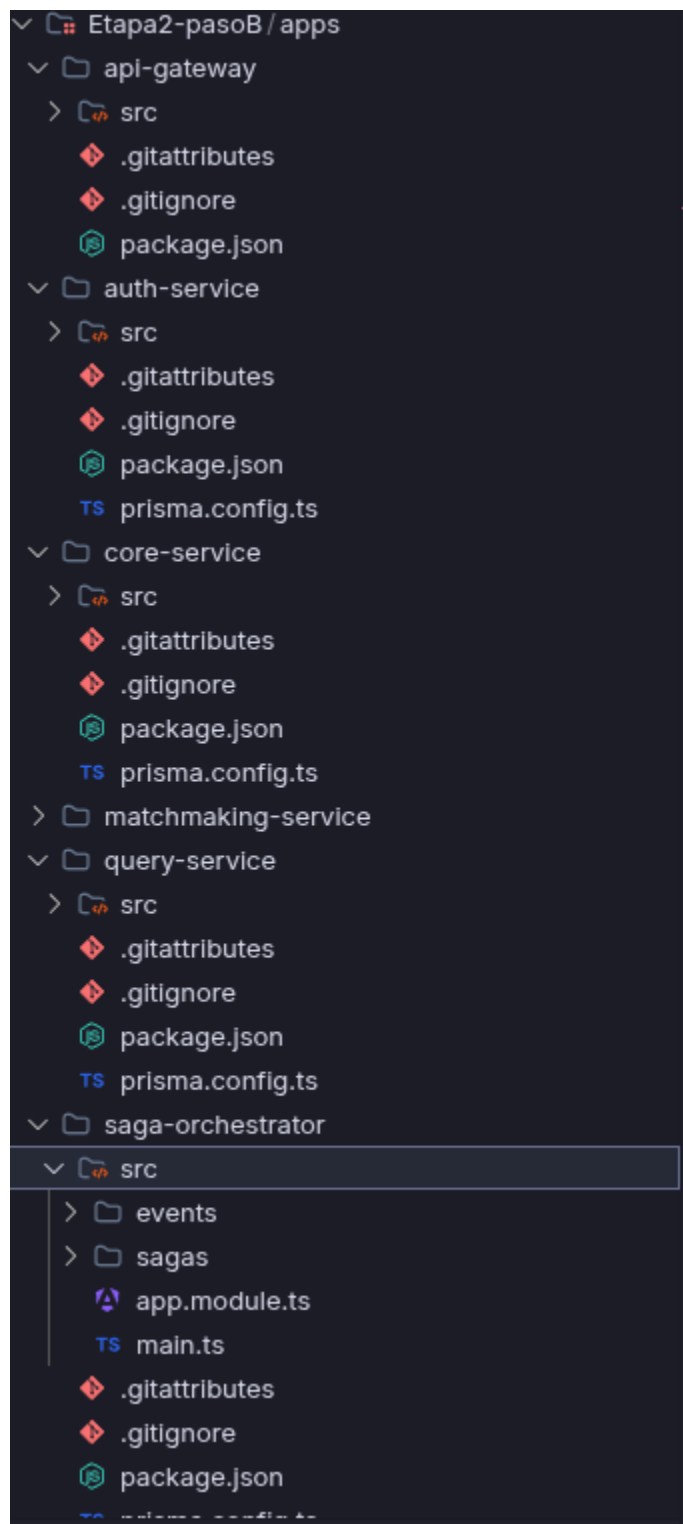
También se separan los esquemas de base de datos y se asigna una base de datos por servicio, se integran circuit breakers para las peticiones HTTP entre servicios y un circuit breaker general para la aplicación.



Paso B — Arquitectura Event-Driven (moderna)

Con la incorporación del servicio de matchmaking, la solución definitiva introduce un Message Broker (RabbitMQ) como intermediario asíncrono y un Saga Orchestrator para coordinar transacciones para evitar datos sucios en la base de datos y un query service para poder hacer consultas rápidas a la base de datos sin necesidad de pasar por los distintos servicios, se pensó esto último para pantallas como el dashboard que muestra turnos del día. El flujo completo, por ejemplo, para la carga de un turno es el siguiente:

- Del lado del cliente, se crea un post /Turno, el api gateway publica un comando en el broker que indica a saga en la base de datos con los datos del turno y el paso actual, el saga devuelve un comando que dispara en el core service la creación del turno, el core service devuelve un evento el cual es escuchado por el Query service que genera un registro en la base de datos plana con los datos de turnos del día. Saga, al estar escuchando los eventos, se da cuenta de que el turno se creó con éxito y arranca la etapa 2 que es pedirle al core service que genere un pago, core genera el pago y devuelve un evento llamado pago creado.
- En caso de alguna falla en el proceso, el saga se encarga de crear una compensación, si saga escucha que el evento disparado por la creación del turno devuelve un resultado inesperado, el mismo activa una compensación para liberar el turno.



FASE 3 Mejora Incremental — Robustez y Escalabilidad

Contexto de crecimiento

La arquitectura Event-Driven de la Fase 2 ya separa responsabilidades entre API Gateway, RabbitMQ, Saga, Query Service y microservicios por dominio. La Fase 3 no cambia esa base: la refuerza para soportar más usuarios, más sedes y picos de demanda en reservas, partidos incompletos y consultas del dashboard.

Mejoras principales

- Redis / ElastiCache: se usa como caché de corta duración para disponibilidad de canchas, partidos incompletos y datos del dashboard. La caché se invalida con eventos del broker, por ejemplo turno.creado, turno.cancelado o partido.actualizado.
- Read Replicas en PostgreSQL: las escrituras críticas siguen yendo al primary, mientras que las lecturas frecuentes se derivan a réplicas. Esto evita que dashboards y búsquedas compitan con las reservas reales.
- Load Balancer y réplicas de servicios: el API Gateway y los microservicios se ejecutan en varias instancias stateless. Si una réplica falla, el balanceador la retira y deriva tráfico a las sanas.
- Auto-scaling: Core, Auth y API Gateway escalan por CPU, memoria y requests; Matchmaking y workers escalan por profundidad de cola; Query Service escala por latencia y cantidad de consultas.

Tolerancia a fallos

Para evitar que una falla parcial afecte todo el sistema, se mantienen los Circuit Breakers para llamadas síncronas residuales y Saga para compensaciones. Además, RabbitMQ incorpora reintentos controlados y Dead Letter Queues para eventos que no pudieron procesarse correctamente.

Descripción del diagrama actualizado

El diagrama de Fase 3 debe agregar un Load Balancer delante del API Gateway, Redis conectado a Query Service y Matchmaking, PostgreSQL Primary + Read Replicas, RabbitMQ en modo cluster, réplicas horizontales por servicio, políticas de auto-scaling, health checks y una DLQ para eventos fallidos.

Justificación técnica y de negocio

Estas mejoras se justifican porque protegen las operaciones centrales del negocio: reservar canchas, completar partidos, cobrar turnos y mostrar información operativa confiable. La solución reduce latencia en horarios pico, mejora disponibilidad y permite crecer sin sobredimensionar infraestructura todo el mes.

FASE 4 Infraestructura, DevOps y Ambientes

Separación de ambientes

La arquitectura de ambientes sigue la progresión estándar de la industria:

Ambiente	Propósito	Validación requerida
Desarrollo	Entorno local + sandbox. Rama: feature/*	Linting, unit tests
Testing / QA	Pruebas funcionales automatizadas. Rama: develop	Suite Mocha+Chai, colecciones Postman/Newman
UAT	Pruebas de humo y pre-producción. Rama: release/*	Smoke tests, validación de stakeholders
Producción	Entorno vivo con usuarios reales. Rama: main	Zero-downtime deploy, rollback automático

Pipeline CI/CD y contenedores

Toda la infraestructura se conteneriza con Docker y se orquesta en Kubernetes (EKS o GKE), con manifiestos de Deployment, Service e Ingress por microservicio. El aprovisionamiento de recursos cloud se gestiona con Terraform (módulos separados por ambiente), garantizando reproducibilidad y auditabilidad.

- Herramienta CI/CD: GitHub Actions con gates de calidad por ambiente
- Ningún artefacto pasa a UAT sin haber superado la suite de pruebas de QA
- Estrategia de branching: GitFlow (feature → develop → release → main)
- IaC: módulos Terraform separados por ambiente con remote state en S3 + DynamoDB lock

Estimación de costos de infraestructura cloud (AWS)

Componente	Servicio Cloud	Cantidad	Rol en la Arquitectura	Costo Mensual (USD)
API Gateway	AWS API Gateway	1	Enrutamiento de entrada	~\$15
Servicio de Reservas	EKS (Kubernetes)	2 pods	Lógica de reservas	~\$80
Servicio Matchmaking	EKS (Kubernetes)	2 pods	Emparejamiento y notificaciones	~\$80
Message Broker	AWS SQS / MSK	1	Comunicación asíncrona	~\$30
Base de Datos	RDS PostgreSQL	1M + 1R	Persistencia de datos	~\$120
Caché	ElastiCache (Redis)	1 cluster	Caché de partidos incompletos	~\$40
Base Vectorial	Pinecone / Chroma	1	RAG / búsqueda semántica	~\$25
CDN / Static	CloudFront + S3	1	Assets estáticos	~\$10
TOTAL ESTIMADO MENSUAL				~\$400

Nota sobre estimación

Los costos son estimaciones aproximadas para un ambiente de producción de escala inicial (hasta ~5.000 usuarios activos/mes). La instancia EC2 se reemplaza por nodos EKS t3.medium. Los precios corresponden a la región us-east-1 y pueden variar según On-Demand vs Reserved Instances.

FASE 5 Extensión de Próxima Generación — Integración con IA

Opción A — Consumo de APIs LLM

En lugar de notificaciones estáticas y genéricas, el sistema integra un modelo de lenguaje (OpenAI GPT-4o o Claude via Anthropic API). El Servicio de Matchmaking, al seleccionar candidatos para un partido, construye un prompt contextualizado con el perfil del jugador (edad, historial, posición preferida, nivel competitivo) y solicita al LLM que redacte una notificación personalizada y persuasiva.

Consideraciones de implementación

Dado que las llamadas al LLM pueden demorar 2–5 segundos, se despachan de forma asíncrona desde el broker, con un timeout configurado y un fallback a notificación estática si se excede el umbral. Se aplica rate limiting para controlar costos por token.

Opción B — Arquitectura RAG (Retrieval-Augmented Generation)

Se implementa un asistente conversacional dentro de la app. El jugador puede escribir o dictar: "Quiero un partido competitivo esta noche cerca del centro". El sistema ejecuta el siguiente pipeline:

- El texto del usuario es convertido a un vector de embeddings (OpenAI Embeddings o sentence-transformers).
- Se realiza una búsqueda semántica por similitud sobre una base de datos vectorial (Pinecone, Milvus o Chroma) que contiene los partidos incompletos embebidos con sus metadatos.
- Los documentos recuperados se inyectan como contexto en el prompt del LLM, que genera una respuesta en lenguaje natural con los partidos más relevantes.

Flujo de ingesta de datos (ciclo de actualización del índice vectorial)

- El Servicio de Matchmaking publica un evento partido.actualizado cada vez que cambia el estado de un partido.
- Un worker dedicado consume el evento, genera el embedding del partido y lo upsertea en la base vectorial.
- El índice permanece sincronizado en tiempo real con el estado del sistema, garantizando que el asistente siempre trabaje con información actualizada.